

DoS Attacks & Network Defenses

Study Notes — Stanford CS Network Security

1. Network Security Guarantees

On-Path vs. Off-Path Attackers

Understanding attacker position is fundamental to threat modeling:

- On-Path Attacker
 - Sits directly on the communication route between sender and receiver.
 - Can read, modify, drop, or replay any packet in transit — full MITM capability.
- Off-Path Attacker
 - Is NOT on the communication route; cannot see traffic passively.
 - Can still launch attacks (e.g., forged packets) but must 'guess' information it cannot observe.
 - **ARP spoofing and BGP hijacking** allow an off-path attacker to move on-path.

What Standard Protocols Lack

Ethernet, IP, UDP, and TCP were designed **for reliability — not security**. They provide no built-in protections:

- Confidentiality
 - All traffic is cleartext. Any on-path observer can read everything.
- Integrity
 - Ethernet, IP, UDP offer zero protection against packet spoofing or modification.
 - TCP gives only a weak source authentication (sequence numbers are guessable).
- Availability
 - Attackers can inject RST (reset) packets to forcibly tear down TCP connections.

Key Takeaway: Always assume the network is malicious. Use TLS whenever you need confidentiality, integrity, or authentication against eavesdropping or on-path modification.

2. Denial of Service (DoS) — Fundamentals

Goal & Core Principle

A DoS attack aims to make a service, network, or organisation unavailable to legitimate users by overwhelming it with traffic or requests.

The attack is efficient because:

- The attacker spends little effort sending cheap/small requests.

- The victim must do expensive processing for each one (computation, memory, bandwidth).

Two Attack Categories

- DoS Bug
 - Exploits a design flaw — usually a protocol asymmetry.
 - Example: client sends a tiny request; server must do expensive work to respond.
 - Or the server must hold state while waiting for the client.
- DoS Flood
 - The attacker controls many machines (botnet) and directs them to send traffic simultaneously.
 - Scale is the weapon — individually harmless packets become devastating in aggregate.

Attack Surface at Every Layer

- Layer 2 / Layer 3 (Data Link / Network):
 - Flood switches/routers beyond their packet-processing capacity.
 - Transport Layer (TCP/UDP):
 - Force the server to maintain a huge number of half-open connections or per-connection state.
 - Application Layer:
 - Trigger expensive database queries, TLS handshakes, or cryptographic computations.
-

3. SYN Flood Attack

TCP Three-Way Handshake — Background

Normal TCP connection setup (needed to understand the vulnerability):

- Client → Server: SYN (with random client sequence number SNc).
- Server → Client: SYN/ACK (with server sequence number SNs). Server stores both SNc and SNs in a half-open connection table.
- Client → Server: ACK. Connection fully established; server removes entry from table.

Vulnerability: The server commits memory to store state (SNc, SNs) BEFORE verifying the client is real. An attacker exploits this gap.

How SYN Flood Works

The attacker (or its bots) sends a massive stream of SYN packets with spoofed (fake) source IP addresses:

- Server receives each SYN and stores a half-open connection entry.
- Server sends SYN/ACKs to the spoofed IPs — those hosts never reply.
- Server's half-open connection (backlog) queue fills up.
- Legitimate clients' SYN packets are now dropped — service is unavailable.

Bad Mitigations & Why They Fail

- Increase backlog queue size:
 - Attacker simply sends more SYNs. Doesn't fix root cause.
- Decrease timeout:
 - Slightly reduces table occupancy but still allocates state — attacker wins with enough rate.

Real Solution: SYN Cookies

The insight: delay state allocation until the 3-way handshake is complete.

How it works:

- Instead of storing SNc/SNs, the server encodes them into the cookie it sends back.
- Cookie formula: $L = \text{MAC_key}(\text{SAddr}, \text{SPort}, \text{DAddr}, \text{DPort}, \text{SNc}, T)$
- The key is random and chosen at boot time. T is a 5-bit counter incremented every 64 seconds.
- Server sets SNs = (T || MSS || L) and sends it in the SYN/ACK.
- An honest client echoes SNs in its ACK. Server recomputes and verifies the MAC.
- Only then does the server allocate socket memory.

Trade-off: SYN cookies lose TCP options negotiated during the handshake (e.g., window scaling, SACK) because the server saved no state.

4. Amplification / Reflection Attacks

Core Mechanism

Any UDP service that responds to a small request with a disproportionately large response can be weaponised:

- Attacker forges the source IP of its UDP packet, setting it to the victim's IP.
- The reflector (DNS server, NTP server, etc.) sends its large response to the victim.
- Attacker needs many reflectors so the victim cannot simply block a handful of IPs.

Why UDP and not TCP? TCP requires a 3-way handshake, so the server would discover the spoofed IP immediately and never send data. UDP has no such check.

Common Amplifiers

- DNS ANY query:
 - Returns ALL records the server has for a domain (A, MX, NS, TXT, etc.).
 - A small ~40-byte query can yield a 3,000+ byte response — ~60-70x amplification.
 - Fix: do not expose recursive resolvers on the public internet.
- NTP MONLIST command:
 - Returns the last 600 clients that queried the server. Enormous amplification factor.
 - Fix: disable MONLIST; considered a misconfiguration today.
 - Reality: hundreds of thousands of misconfigured hosts still exist publicly.

Real-World Scale

- 2013: 300 Gbps DNS amplification attack — 31,000 open resolvers across 3 networks that permitted IP spoofing.
 - 2014: 400 Gbps DDoS using 4,500 misconfigured NTP servers.
 - 2018: Memcached amplification reached 1.7 Tbps (see Memcached section).
-

5. Botnets & the Mirai Malware

October 2016 — Dyn Attack

On October 21, 2016, a massive DDoS attack against Dyn (a major DNS provider) took down Twitter, Netflix, PayPal, Spotify, GitHub, Reddit, and others for most of the day. Estimates: up to 100,000 malicious endpoints, possibly 1.2 Tbps peak.

How the IoT Botnet Worked

The attack used ~200,000 compromised IoT devices (cameras, routers, baby monitors, printers). Unlike amplification, this was a direct flood:

- Devices sent SYN, ACK, UDP, and GRE packets directly at targets — no spoofing needed.
- Volume was the weapon: 200K devices each contributing even modest bandwidth adds up fast.

Mirai Malware Architecture

Mirai is the malware family responsible. Its C2 infrastructure:

- Bots (infected devices) continuously scan random IPs for open Telnet/SSH with default passwords.
- Newly found devices are reported to a Report Server.
- A Loader binary is dispatched to infect the new device.
- The Bot Master sends Attack Commands specifying: Action (START/STOP), Target IPs, Attack Type (GRE/DNS/TCP), Duration.

Why Mirai Was Successful

Mirai's code is poorly written and uses no novel techniques. Success came from:

- IoT security is virtually non-existent — devices shipped with hardcoded default credentials (e.g., 'admin/admin', '123456').
- Attack simplicity meant the malware ran on heterogeneous hardware (ARM, MIPS, x86, etc.).
- Stateless scanning (based on ZMap techniques) let it enumerate the internet rapidly without maintaining per-target state.

Example credentials targeted: Passwords like 123456, 888888, admin, pass, root, guest — often hardcoded by manufacturers across many device types (Dahua DVRs, HiSilicon cameras, Toshiba cameras, Ubiquiti routers, Xerox printers, etc.).

6. Memcached Amplification (2018)

Memcached is a distributed caching system. Attackers discovered that a tiny UDP request asking to retrieve a large cached record would cause the server to return up to 50,000x the data received.

- The attacker spoofs the victim's IP as the source address on the UDP request.
- The Memcached server floods the victim with the enormous cached record response.
- Works only over UDP — TCP would require a handshake revealing the spoofed source IP.
- Peak attack size reached 1.7 Tbps (as of March 2018, per NETSCOUT/Arbor data).

Analogy: Like calling a restaurant and saying 'read me the entire menu, and call me back at [victim's number]'.

7. Defenses Against DDoS

Google Project Shield

A free service providing DDoS protection for news sites, human rights organisations, and election monitoring sites — groups that are commonly targeted for censorship via DDoS.

- Uses Google's massive bandwidth and infrastructure as a protective proxy.
- The proxy absorbs the flood of SYNs and SYN/ACKs; only legitimate ACKs (completed connections) are forwarded to the real website.
- Effective because Google's capacity exceeds what most botnets can saturate.

Moving Up the Stack: GET Floods

Once flood protection is in place at the packet level, attackers escalate to the application layer:

- Bots complete the full TCP handshake (so source IPs are real and traceable).
- Bots complete the TLS handshake (expensive cryptographic operations for the server).
- Bots issue HTTP GET requests for large resources (big images, files) — draining bandwidth.
- Downside for attacker: real source IPs are exposed, so the victim can block or rate-limit those IPs.

Ingress Filtering (BCP 38)

Root cause of amplification attacks: ISPs allow packets with spoofed source IPs to leave their networks. Ingress filtering fixes this:

- An ISP checks every outgoing packet to verify its source IP belongs to a prefix actually assigned to that customer.
- If not, the packet is dropped before it reaches the internet.

Why it is hard to deploy:

- Requires global adoption — if even 10% of networks skip it, amplification attacks remain viable.
- No direct incentive for ISPs: spoofed packets harm others, not the ISP's own customers.
- As of 2017 (CAIDA data): 33% of autonomous systems and 23% of announced IP space still allow spoofing.
- The 2013 300 Gbps attack originated from only 3 networks that permitted IP spoofing.

Booster / Stresser Services

DDoS-as-a-service is commercially available. Prices as low as ~\$24/month for plans providing 220 Gbps capacity with 24/7 support. Payment via PayPal or Bitcoin. Lowered the barrier for unskilled attackers dramatically.

Github Attack (2018) — JavaScript-Based DDoS

A 1.35 Tbps DDoS against GitHub used a novel method:

- Malicious JavaScript was injected into regular HTTP web page responses by a Man-in-the-Middle entity.
 - Every ordinary user who visited an affected page unknowingly had their browser send requests to GitHub.
 - No botnet or compromised device needed — legitimate users became unwitting weapons.
 - The Chinese government was widely suspected as the operator ('Great Cannon').
-

8. Building Safe Network Protocols

Designing network protocols from scratch is hazardous. Parsing network packets is an extremely error-prone task that has historically led to countless vulnerabilities.

Best practices:

- Never roll your own crypto — use well-audited primitives.
- Use established RPC frameworks instead of custom binary protocols.

Recommended: gRPC (HTTP/2 + TLS 1.3)

- Supports 11 languages with safe, generated parsers — eliminates manual packet parsing.
- Exceptionally efficient binary wire format (Protocol Buffers).
- Supports streaming, synchronous, and asynchronous call patterns.
- TLS-based mutual authentication built in.

Principle: Prefer proven building blocks (gRPC, TLS, established protocols) over custom solutions. Security bugs in custom parsers are common and severe.